

Génie logiciel



Méthodes et outils

Patrice EFFI BROU

2020

Sommaire

INTRODUCTION

Le génie logiciel désigne l'ensemble des méthodes techniques et outils concourant à la production d'un logiciel. Le génie logiciel n'est pas de la programmation qui est faite par une seule personne et n'utilise pas de méthodes complexes comme c'est le cas en génie logiciel.

OBJECTIFS :

- Préparer aux projets de fin de stage
- Changer la manière de travailler
- connaître les bonnes pratiques en terme d'architecture
- pratiquer les tests de logiciel
- planifier un projet de production de logiciel

CHAPITRE I : GENERALITES

1.1 MOTIVATION A LA CREATION DU GENIE LOGICIEL

Le génie logiciel (1968) a été créé dans le souci de trouver des méthodes concernant le travail pour gérer le coût de la création des logiciels. Un projet peut coûter plus des centaines de millions voir des milliards de CFA, il faut donc s'assurer que l'on a de très fortes chances de réussir le projet en maîtrisant tous les facteurs de risques.

Les dix éléments qui peuvent causer l'échec d'un projet logiciel en cas de défaillance sont les suivants :

- Le soutien des dirigeants
- L'implication des utilisateurs
- L'expérience du gestionnaire de projet
- Définition des objectifs
- Les coûts
- L'infrastructure (architecture logicielle)
- Les exigences matérielles et logicielles
- La méthodologie utilisée
- Les estimations (coûts, délais, personnel etc...)
- Autres

Source: The Standish Group report, The chaos ten , 2000

1.2 COMMENT PRODUIRE DES LOGICIELS

Suite à la crise du logiciel (1970) et Inspiré du génie civil, *le génie logiciel est un ensemble coordonné d'activités régulées dont le but est de fabriquer un produit logiciel*. Une activité en informatique (dans le processus de développement) dit ce qu'il faut faire et quelle activité faire en premier.

121 la règle du CQFD

Le GL se préoccupe des procédés de fabrication des logiciels de façon à s'assurer que les 4 critères suivants soient satisfaits.

- Le système qui est fabriqué répond aux besoins des utilisateurs (correction fonctionnelle).
- La qualité correspond au contrat de service initial. La qualité du logiciel est une notion multiforme qui recouvre:
 - la validité : aptitude d'un logiciel à réaliser exactement les tâches définies par sa spécification,
 - la fiabilité: aptitude d'un logiciel à assurer de manière continue le service attendu,
 - la robustesse : aptitude d'un logiciel à fonctionner même dans des conditions anormales,
 - l'extensibilité : facilité d'adaptation d'un logiciel aux changements de spécification,
 - la réutilisabilité : aptitude d'un logiciel à être réutilisé en tout ou partie,
 - la compatibilité : aptitude des logiciels à pouvoir être combinés les uns aux autres,
 - l'efficacité : aptitude d'un logiciel à bien utiliser les ressources matérielles telles la mémoire, la puissance de l'U.C., etc.

- la portabilité : facilité à être porté sur de nouveaux environnements matériels et/ou logiciels,
- la traçabilité : capacité à identifier et/ou suivre un élément du cahier des charges lié à un composant d'un logiciel,
- la vérifiabilité : facilité de préparation des procédures de recette et de certification,
- l'intégrité : aptitude d'un logiciel à protéger ses différents composants contre des accès ou des modifications non autorisés,
- la facilité d'utilisation, d'entretien, etc.
- Les coûts restent dans les limites prévues au départ.
- Les délais restent dans les limites prévues au départ

la norme ISO 25010 de la série de normes ISO 250xx, également appelée SQuaRE (pour *software quality requirements and evaluation* en anglais, c'est-à-dire exigences et évaluation de la qualité du logiciel).

122 Les activités classiques de développement génie logiciel

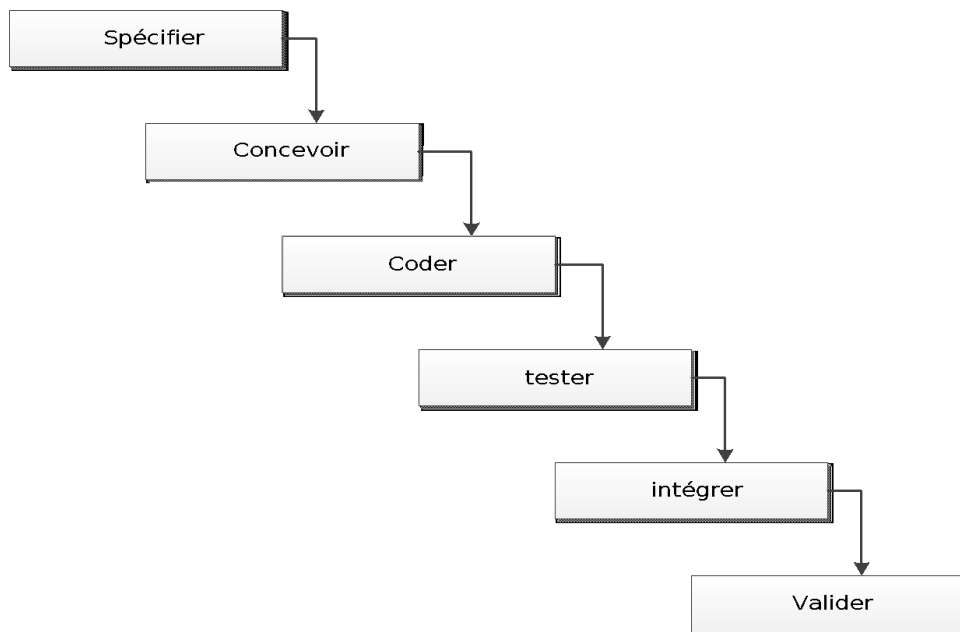
SDLC est une abréviation de Software Development Life Cycle. Le **cycle de vie de développement logiciel** est une série d'étapes qui offre un modèle défini pour le développement et la gestion du cycle de vie d'une application. Les activités classiques de développement logiciel sont :

- (1) **Spécifier** (spécification) : Qu'est-ce que le logiciel doit permettre de faire ?
- (2) **Concevoir** (conception)
 - Conception architecturale** : Comment installer le logiciel et sur combien de serveurs ? Sous quel système d'exploitation et quel type d'ordinateur ?
 - Conception de l'organisation des données** : Comment les données sont-elles organisées sur le disque dur ?
 - Conception de l'organisation des programmes** : Penser à l'environnement dans lequel le logiciel va être exécuté (JAVA, WEB, C++, etc)
- (3) **Coder** (codage) : Ecrire le programme dans le langage choisi
- (4) **Valider** : Comparer ce que fait le logiciel avec ce qui a été spécifié. Est-ce qu'il fait ce qu'on attend de lui ?
- (5) **Intégrer** : Bien organiser le programme pour qu'il fonctionne dans son environnement
- (6) **Tester** : Pour vérifier si les codes sont bien écrits

1. 3 ETUDE DE QUELQUES MODELES DE CYCLE DE VIE

131 Modèle en cascade (waterfall)

C'est un processus de six étapes appelé la cascade. Il est séquentiel



Les vérifications se font avec celui qui a commandé le logiciel

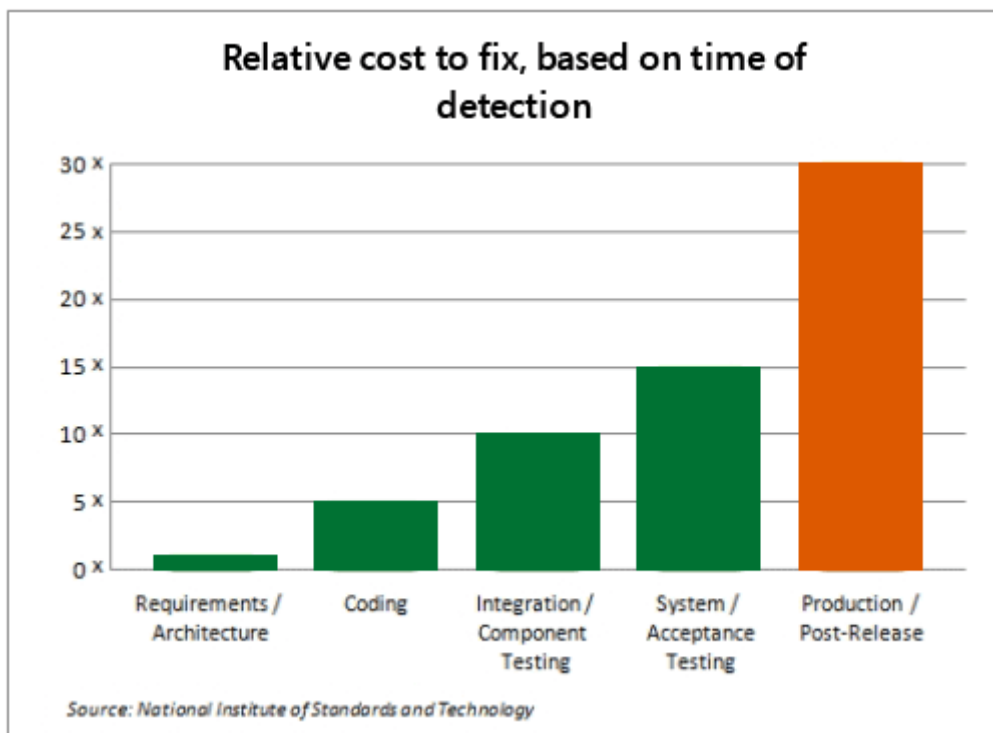
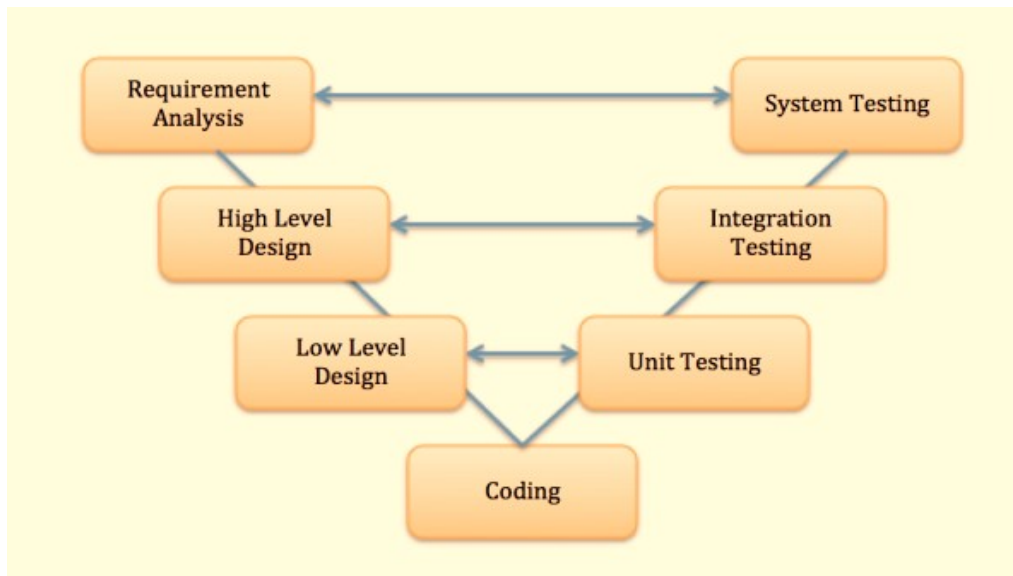


Fig x : *Représentation du coût de correction d'un Bug en fonction du moment de sa détection*

132 Modèle en V

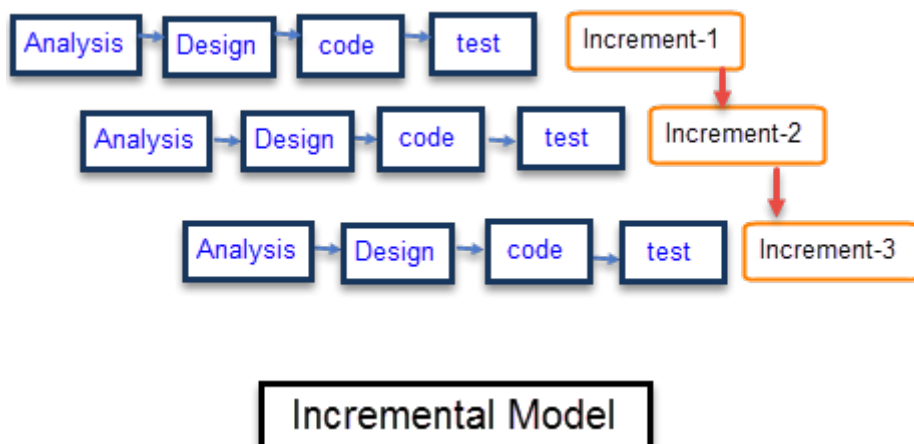


Il intègre des vérifications à chaque niveau, mais comme dans le cas du modèle en cascade les utilisateurs attendent à la fin de la fabrication. Cela pose des problèmes quand le projet est volumineux. Ici les utilisateurs finaux ne participent pas à la conception.

133 Processus itératif et incrémental

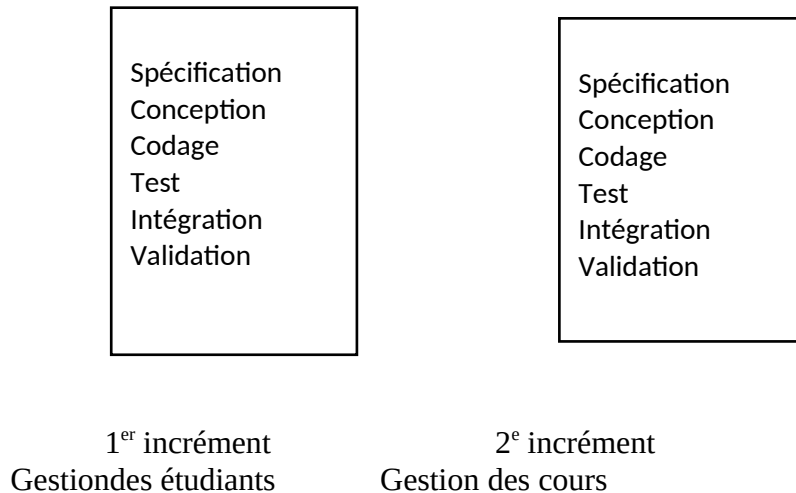
Qu'est-ce que le modèle incrémental?

Le modèle incrémental est un processus de développement logiciel où les exigences sont décomposées en plusieurs modules autonomes du cycle de développement logiciel. Le développement incrémentiel se fait par étapes depuis la conception de l'analyse, la mise en œuvre, les tests / vérification et la maintenance.



La méthode est itérative et incrémentale. On choisit un module (une partie du travail à faire) et on le réalise puis on passe au suivant.

Exemple : Ecole



Avantages

Les choix techniques sont validés très tôt par les tests de validation et le logiciel est vérifié également très tôt. Cela permet de satisfaire partiellement dans un bref délai les attentes des utilisateurs.

134 Modèle en spirale

Le modèle en spirale a été défini par Barry Boehm en 1988 dans son article "A Spiral Model of Software Development and Enhancement". Ce cycle ressemble beaucoup au cycle en V, mais ce cycle permet mieux de gérer les risques à chaque itération. On n'a pas la possibilité de retourner en arrière.

Quelques risques à évaluer

1. La défaillance de personnel: embauche de haut niveau, formation mutuelle, leaders, adéquation profil/fonction, ...
2. Le calendrier et budgets irréalistes: estimation détaillée, développement incrémental, réutilisation, élagage des besoins, ...
3. Le développement de fonctions inappropriées: revues d'utilisateurs, manuel d'utilisation précoce, ...
4. Le développement d'interfaces utilisateurs inappropriées: maquettage, analyse des tâches,

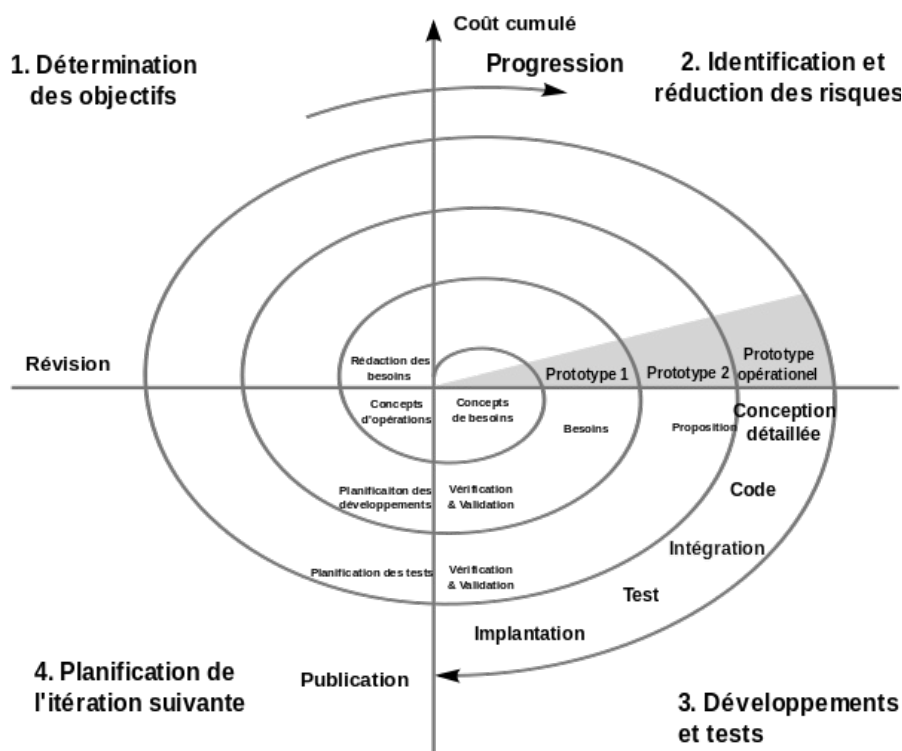
5. Le produit «plaqué or»: analyse des coûts/bénéfices, conception tenant compte des coûts, ...

6. La volatilité des besoins : développement incrémental de la partie la plus stable d'abord, masquage d'information, ...

7. Les problèmes de performances : simulations, modélisations, essais et mesures, maquettage,

8. les exigences démesurées par rapport à la technologie : analyses techniques de faisabilité, maquettage, ...

9. les tâches ou composants externes défaillants : audit des sous-traitants, contrats, revues, analyse de compatibilité, essais et mesures, ..



Cette méthode constitue quatre phases.

1. Détermination des objectifs.

Ici on détermine les différents objectifs à attendre dans l'itération courante.

2. Analyse et résoudre les risques.

On analyse les différents risques causé par cette itération.

3. Développement et vérification de la solution pour planifier la phase suivante.

Commencer les développements de la solution en question.

4. Revue des résultats et vérification du cycle pour valider le développement.

La recette permet de s'assurer formellement que le produit est conforme à l'objectif.

Properties of Model	Water-Fall Model	Incremental Model	Spiral Model	Rad Model
Planning in early stage	Yes	Yes	Yes	No
Returning to an earlier phase	No	Yes	Yes	Yes
Handle Large-Project	Not Appropriate	Not Appropriate	Appropriate	Not Appropriate
Detailed Documentation	Necessary	Yes but not much	Yes	Limited
Cost	Low	Low	Expensive	Low
Requirement Specifications	Beginning	Beginning	Beginning	Time boxed release
Flexibility to change	Difficult	Easy	Easy	Easy
User Involvement	Only at beginning	Intermediate	High	Only at the beginning
Maintenance	Least	Promotes Maintainability	Typical	Easily Maintained
Duration	Long	Very long	Long	Short
Risk Involvement	High	Low	Medium to high risk	Low
Framework Type	Linear	Linear + Iterative	Linear + Iterative	Linear
Testing	After completion of coding phase	After every iteration	At the end of the engineering phase	After completion of coding
Overlapping Phases	No	Yes (As parallel development is there)	No	Yes
Maintenance	Least Maintainable	Maintainable	Yes	Easily Maintainable
Re-usability	Least possible	To some extent	To some extent	Yes
Time-Frame	Very Long	Long	Long	Short
Working software availability	At the end of the life-cycle	At the end of every iteration	At the end of every iteration	At the end of the life cycle

Properties of Model	Water-Fall Model	Incremental Model	Spiral Model	Rad Model
Objective	High Assurance	Rapid Development	High Assurance	Rapid development
Team size	Large Team	Not Large Team	Large Team	Small Team
Customer control over administrator	Very Low	Yes	Yes	Yes

135 Les methodes agiles

Elles introduisent une nouvelle approche pour le développement de produit en remplaçant, le client au cœur de l'action et en prônant l'adaptation des procédés de création au fil de l'évolution du projet.

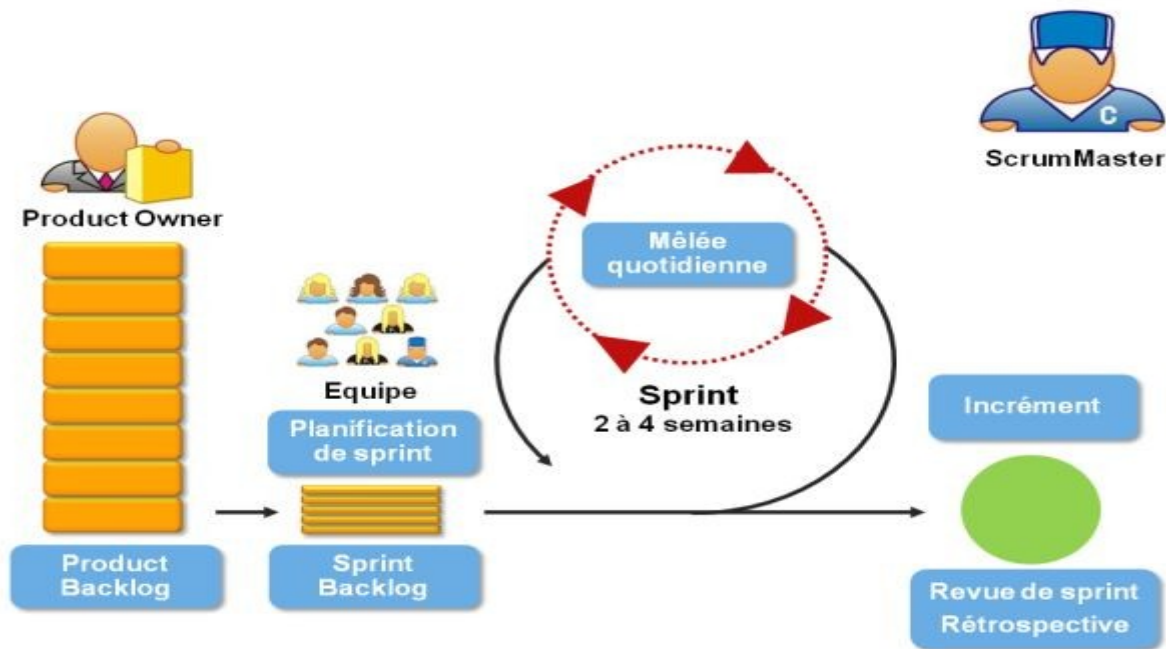
Scrum et Safe sont les méthodes agiles les plus utilisées. Selon [le benchmark VersioOne daté de 2019](#), Scrum pèse 54% de parts de marché dans l'agilité mono-équipe, et Safe 30% dans l'agilité multi-équipes.

SCRUM est une méthode de développement agile qui se concentre spécifiquement sur la façon de gérer les tâches dans un environnement de développement en équipe.

Fondamentalement, Scrum est dérivé d'une activité qui se produit pendant un match de rugby. Scrum croit en l'autonomisation de l'équipe de développement et préconise le travail en petites équipes (disons 7 à 9 membres). Il se compose de trois rôles et leurs responsabilités sont expliquées comme suit:

Scrum définit seulement 3 rôles :

- Le Product Owner qui porte la vision du produit à réaliser et travaille en interaction avec l'équipe de développement. Il s'agit généralement d'un expert du domaine métier du projet.
- L'Equipe de Développement qui est chargée de transformer les besoins exprimés par le Product Owner en fonctionnalités utilisables. Elle est pluridisciplinaire et peut donc encapsuler d'autres rôles tels que développeur, architecte logiciel, DBA, analyste fonctionnel, graphiste/ergonome, ingénieur système.
- Le Scrum Master qui doit maîtriser Scrum et s'assurer que ce dernier est correctement appliqué. Il a donc un rôle de coach à la fois auprès du Product Owner et auprès de l'équipe de développement. Il doit donc faire preuve de pédagogie. Il est également chargé de s'assurer que l'équipe de développement est pleinement productive. Généralement le candidat tout trouvé au rôle de Scrum Master est le chef de projet. Celui ci devra cependant renoncer au style de management « commander et contrôler » pour adopter un mode de management participatif.



Pré requis recommandés

- Un grand mur libre et dégagé dans l'espace de travail de l'équipe.
- Blocs de post-it et marqueurs.
- L'ouvrage « Scrum et XP depuis les tranchées ».
- logiciel de Planning .

Elles se sont depuis imposées comme des standards sur le terrain du développement d'application.

eXtreme Programming (XP)

La technique **eXtreme Programming (XP)** est très utile lorsqu'il y a des demandes ou des exigences en constante évolution de la part des clients ou lorsqu'ils ne sont pas sûrs de la fonctionnalité du système. Il préconise des «versions» fréquentes du produit dans des cycles de développement courts, ce qui améliore intrinsèquement la productivité du système et introduit également un point de contrôle où toutes les exigences des clients peuvent être facilement mises en œuvre. Le XP développe des logiciels en gardant le client dans la cible.

Safe Scaled Agile [Framework](#) for Lean software and Systems engineering

Définition

La spécification des besoins est un contrat entre les futurs utilisateurs et les concepteurs. Elle concerne les caractéristiques attendues du futur système (logiciel). On peut aussi spécifier l'architecture d'un logiciel. En phase de programmation, les ingénieurs concepteurs spécifient les programmes à l'attention des utilisateurs. Une spécification doit être claire et précise. Il faut penser à la complétude. C'est-à-dire se poser la question « est-ce qu'on a pensé à tous les cas? ». Par exemple pour un système de résolution d'équation de second degré dans $\mathbb{R}^2$, le cas où tous les coefficients sont nuls ne doit pas être oublié.

I- TYPLOGIE DES SPECIFICATIONS

Les spécifications se font selon deux critères :

- La formalité :

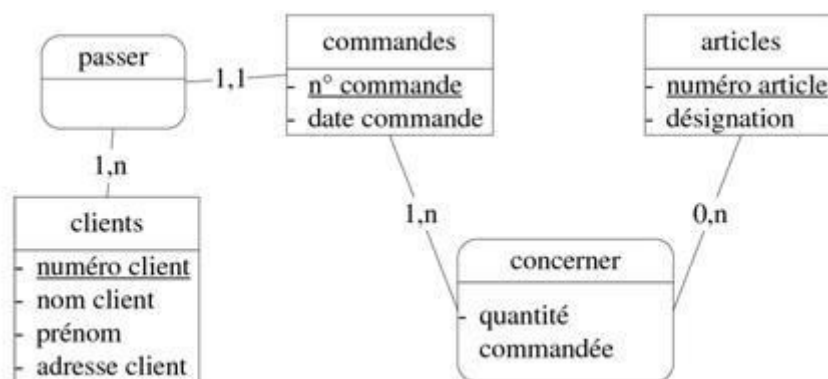
Il Ya des spécifications informelles (ou non formelles) en langage naturel, des spécifications semi-formelles et des spécifications formelles.

- Le caractère opérationnel ou déclaratif

Le opérationnel concerne le comportement du système, et Le caractère déclaratif concerne les propriétés du système.

Exemples

- a) Le modèle entité association est de caractère semi-formel et déclaratif.



- b) L'algorithme suivant :

Pour i 1 à 100

Lire (note)

Si $note < 10$ alors résultat 'refusé'

Sinon résultat 'admis'

Fin pour

C'est une spécification semi-formelle opérationnelle. Il s'agit du mode opératoire du calcul de résultats de fin d'année.

II- TECHNIQUES DE SPECIFICATIONS EN PHASE D'ANALYSE (CONCEPTION)

1- Spécification en langage naturel

Les techniques les plus utilisées sont les techniques de spécification en langage naturel. Mais cela constitue l'une des principales causes d'échec des projets car cette technique manque de structuration et peut contenir des ambiguïtés. L'une des solutions serait de penser à la complétude et rédiger les solutions dans un cahier de charges à valider par les parties contractuelles.

SOMMAIRE

1. Présentation de l'entreprise
 - A.1. Les objectifs
 - A.2. Les cibles
2. Les concurrents
3. Développement du logiciel
 - C.1. Caractéristiques et fonctionnalités
 - C.2. Structure du logiciel
4. Charte graphique du logiciel
5. Les spécificités et les livrables
 - E.1. Les contraintes techniques
 - E.2. Les livrables
 - E.3. Le planning

2- Spécification technique dans un langage évolué

On insère des commentaires dans le code. Le programmeur pourra à partir de ces spécifications écrire le code de l'implémentation. Pratiquement tous les langages évolués permettent de réaliser ces spécifications. PASCAL, JAVA, ADA, C++, etc.
Des outils complémentaires peuvent être utilisés pour générer la documentation dans des divers formats HTML, PDF

L outil JAVADOC intégré au SDK java permet de générer la documentation qui peut être utilisé comme aide ou manuel de spécification. Le tableau ci-dessous montre l'intégration des commentaires dans un programme écrit en java et la documentation générée au format html.

Avec Java Doc, on génère la documentation

```
public class Cacao {  
  
    /**  
     * @param args  
     */  
    /**@author pbrou  
     @version 3.0  
     */  
    /** Procédure qui affiche le reste des fèves traitées */  
    public void reste_fève() {  
  
        int volu;    // Le volume de fève traité  
        // To do  
  
    }  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
  
    }  
}
```

Class Cacao java.lang.Object

└ Cacao

Constructor Summary

Cacao()	
-------------------------	--

Method Summary

static void	main (java.lang.String[] args)
void	reste_fève () Procédure qui affiche le reste des fèves traitées

Constructor Detail

Cacao

public Cacao()

Method Detail

reste_fève

public void **reste_fève**()

Procédure qui affiche le reste des fèves traitées

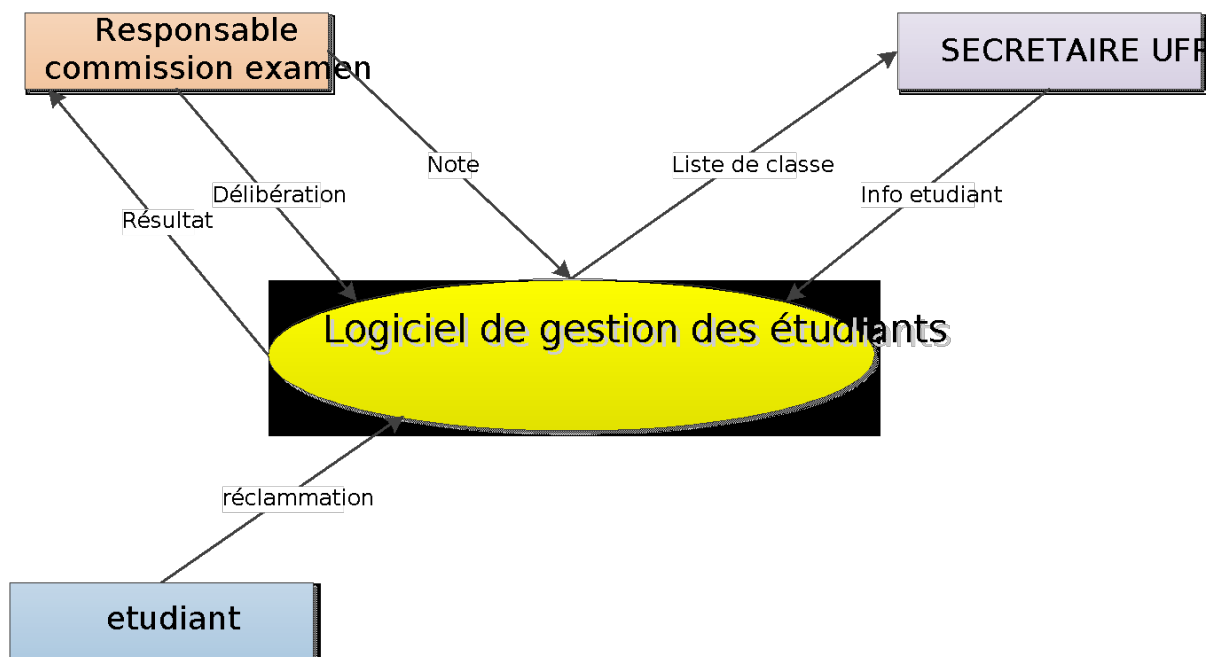
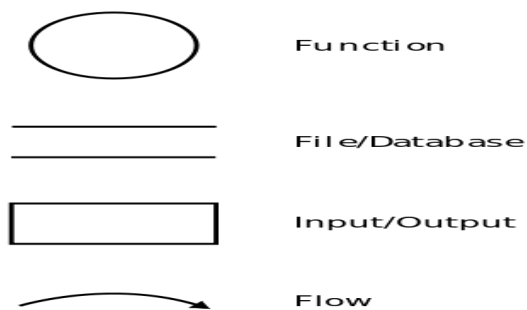
3 Le diagramme de flot de données

C'est une technique semi-formelle opérationnelle.

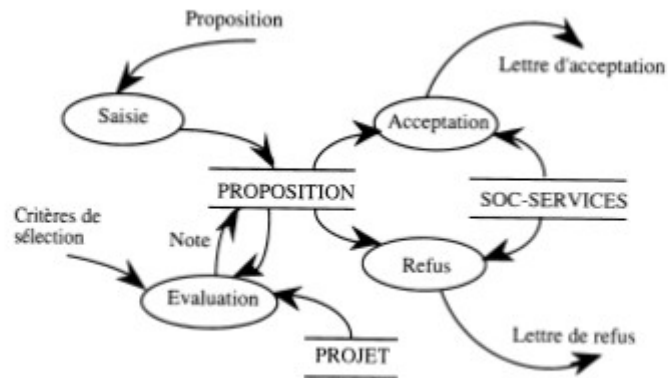
Le formalisme pour faire le diagramme de flot de données :

Les DFD décrivent des collections de données manipulées par des fonctions. Les données peuvent être persistantes (dans des stockages) ou circulantes (représentées par des flots de données).

Il y'a deux(2) niveaux d'abstractions (niveau conceptuel, niveau détaillé)



DFD conceptuel



DFD détaillé

NB : il y'a d'autres techniques de spécifications telles que : les automates à état finis(AF), les réseaux de pétri, le modèle entités associations(E/A).

Il y'a également plusieurs techniques de spécifications formelles :

- ✓ Notation Z
- ✓ OCL(Object Constraints Language)
- ✓ etc.

CHAPITRE 3 : LES TECHNIQUES DE CONCEPTION

Une fois le problème spécifié, il faut le concevoir sur le plan architectural (architectures logiciel et matérielle) et donner la description détaillée des modules.

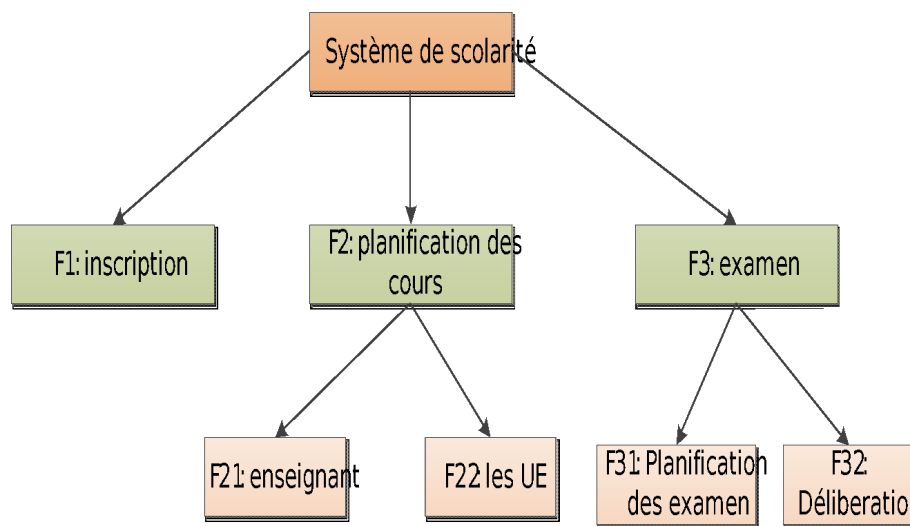
- Architecture logicielle
- Architecture matérielle
- Fonction détaillée

31 Approche fonctionnelle

Elle utilise la notion de modules.

Un module est un composant d'une application contenant des définitions de données et des fonctions et constituant un tout cohérent.

Les modules peuvent être reliés par l'intermédiaire de relations ou peuvent être utilisés par d'autres modules.



Système de gestion de scolarité

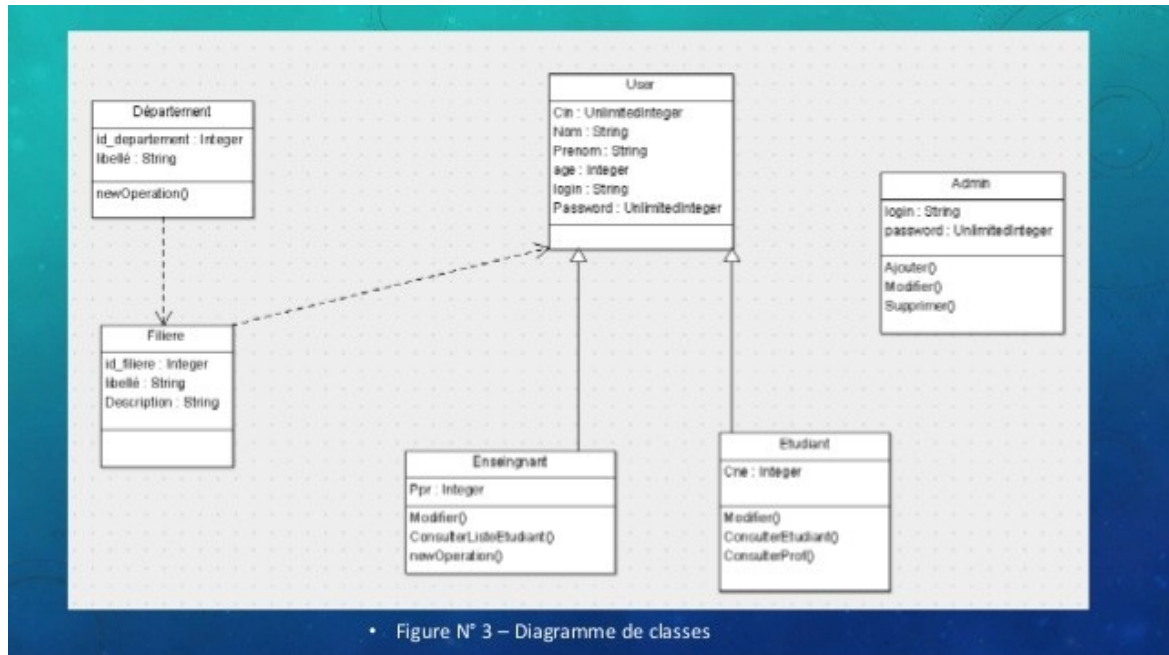
Les méthodes d'approche fonctionnelles SADT (1970) permettent de représenter la hiérarchie des fonctions.

32 L'approche objet

Dans l'approche objet, les modules principaux correspondent à des objets concrets ou abstraits du domaine de l'application (objet métier). Les objets métiers sont incontournables, il ne faut donc pas les oublier.

Pour le système correspondant, les objets métiers sont :

- Etudiant
- Filière
- Matière(UE)
- Session
- Enseignant



Les objets sont décrits dans des classes dans la plupart des langages orienté- objets (C++, C#, JAVA, PHP, VB, VB.NET, etc.).

La notation UML permet de faire la conception objet.

33 L'approche Systemique : MERISE

34 Conception architecturale

Toute application peut être découpée selon les 3 niveaux :

- L'interface avec l'utilisateur

Concerne le système de disposition des éléments graphique (bouton, label, ...)

- Les traitements

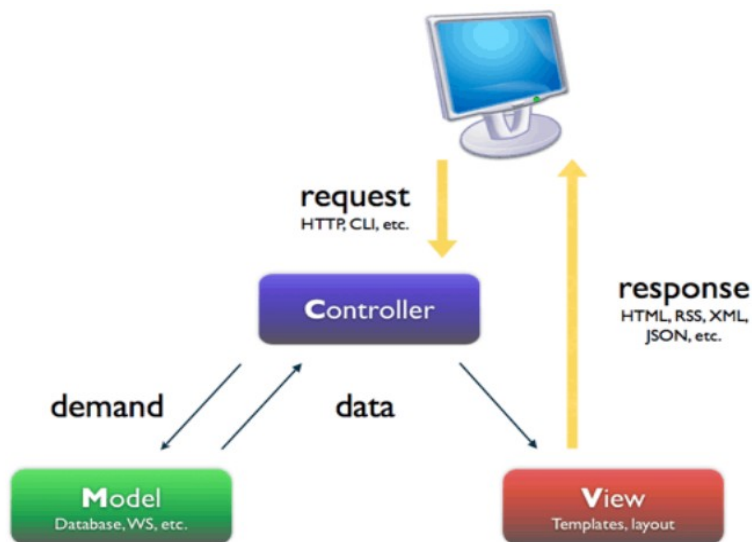
Concerne les algorithmes et les requêtes SQL de manipulation des données

- Les données

Concerne le contenu de la BD que l'on consulte pour les manipulations et les mises à jour.

Exemple : Le modèle MVC

- Le **Modèle** (pour les données) : il fait le lien entre l'application et la BD
- La **Vue** (pour l'interface utilisateur) : elle contient l'ensemble des fenêtres de l'application.
- Le **Contrôleur** (pour le traitement des requêtes de l'utilisateur) : c'est lui qui, selon la requête de l'utilisateur, va chercher les données (chez le Modèle), remplit la vue correspondante et l'affiche à l'utilisateur.



le modele MVC peut s'implémenter une meme machine.

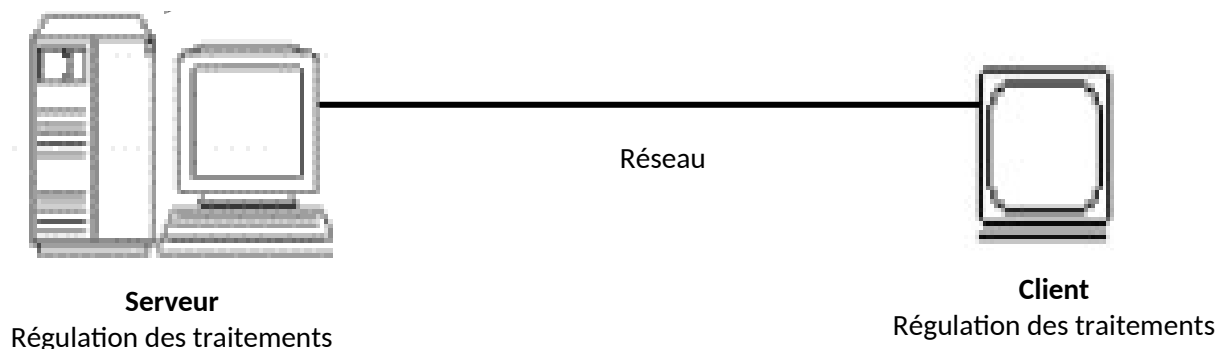
application n tier

Dans une architecture centralisé, l'ordinateur central est bloquant, il n'y a pas d'autonomie des terminaux (0% traitement, affichage).

Pourquoi ne pas décentraliser les traitements ?

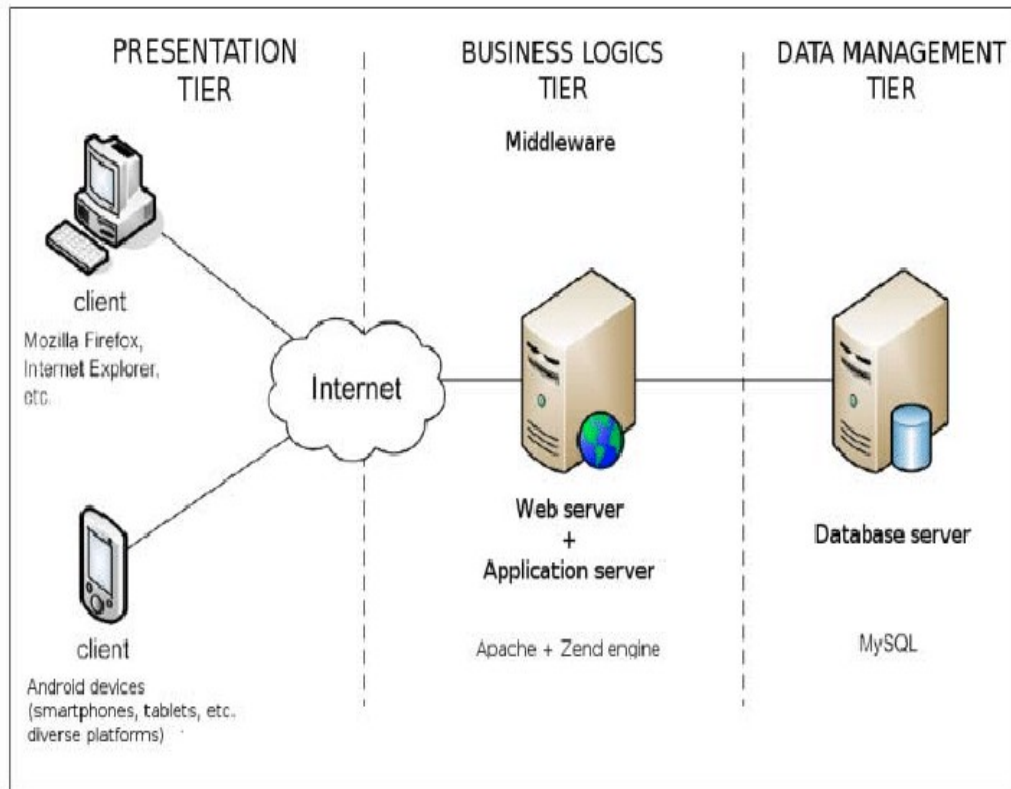
La solution c'est le client/serveur dont les principes généraux sont :

- Des postes de travail multifonction. Ils peuvent se connecter à plusieurs serveurs.
- Les traitements peuvent être répartis entre serveur et client



Le client/serveur dans une entreprise

La repartition des composants sur des machines ou des serveurs différents donnent des architectures n tier.



les web services

Qu'est-ce qu'un Web service?



Par définition, un service Web est un logiciel qui se rend disponible sur Internet et normalise sa communication via le codage XML. Un client appelle un Web service en envoyant une requête (généralement sous la forme d'un message XML, Json ou HTTP), et le service renvoie une réponse sous forme XML, Json ou HTTP.

Les services Web invoquent la communication sur un réseau, HTTP étant le moyen de connectivité le plus courant entre deux systèmes. L'architecture SOA (Services Oriented Architecture) s'appuie sur des services Web et reposent principalement sur des normes telles que XML-RPC et SOAP (Simple Object Access Protocol).

L'une des principales critiques de Web service est le degré de couplage entre le client et le serveur.

Chapitre IV LES OUTILS

41 la planification

WBS (work break down structure)

GANTT

PERT

42 L'évaluation

COCOMO II

43 la gestion des versions

44 les outils Case (AGL atelier de genie logiciel)